文件上传漏洞

介绍

文件上传漏洞（File Upload Vulnerability）是指Web应用程序在允许用户上传文件时，未对文件类型、内容、大小等进行严格校验，导致攻击者能够上传恶意文件（如WebShell、病毒、木马等），从而获取服务器控制权或执行其他恶意操作的安全漏洞。

```
+-----------------------+
|   恶意用户（攻击者）     |
+----------+------------+
           |
           | 1. 上传恶意文件
           v
+-----------------------+
|      Web应用程序        |
|                       |
| 2. 未严格校验文件类型   |
|   或内容，允许上传      |
+----------+------------+
           |
           | 3. 文件存储到服务器
           v
+-----------------------+
|        服务器文件系统     |
|                       |
|   uploads/shell.jsp   |
+----------+------------+
           |
           | 4. 攻击者访问恶意文件
           v
+-----------------------+
|     执行恶意代码         |
| (如RCE、数据泄露等)     |
+-----------------------+
```

柔韵安全内部培训资料

简单漏洞Demo

我们先来实现个简单的文件上传功能

```
@MultipartConfig
@WebServlet("/upload")
public class UploadController extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException, ServletException {
        // 获取 webapp 目录下的 uploads 文件夹路径
        String rootPath = getServletContext().getRealPath("/");
        File uploadDir = new File(rootPath + File.separator + "uploads");

        if (!uploadDir.exists()) {
            uploadDir.mkdirs();
```

```
        }

        // 获取上传的文件 Part 对象
        Part filePart = request.getPart("file");
        String fileName = extractFileName(filePart);

        if (fileName == null || fileName.isEmpty()) {
            response.getWriter().write("上传失败：文件为空");
            return;
        }

        // 构建目标文件路径
        File dest = new File(uploadDir.getAbsolutePath() + File.separator +
fileName);

        // 写入文件到服务器
        filePart.write(dest.getAbsolutePath());

        // 返回响应
        response.getWriter().write("上传成功，文件路径：" + dest.getAbsolutePath());
    }

    // 从 Part 中提取原始文件名
    private String extractFileName(Part part) {
        String contentDisp = part.getHeader("content-disposition");
        for (String cd : contentDisp.split(";")) {
            if (cd.trim().startsWith("filename")) {
                return cd.substring(cd.indexOf('=') + 1).trim().replace("\"",
"");
            }
        }
        return null;
    }
}
```

这是个非常简单的文件上传代码，从获取文件到上传整个过程中只有判断文件是否上传，那么存在漏洞是必然的

这里我提一嘴，任意文件上传和任意文件上传getshell是两个东西，这个是不一样的，在cve中也有两个对应条目

任意文件上传和任意文件上传getshell的区别

任意文件上传（Arbitrary File Upload）

- **定义**：攻击者可以上传任意类型文件到服务器，但**不直接导致代码执行。**
- **CVE案例**：
  - **CVE-2021-42013**（Apache HTTP Server）：允许上传恶意文件，但需结合其他漏洞才能执行。
  - **CVE-2022-29464**（WSO2 API Manager）：上传.jsp文件但未自动解析。
- **危害**：
  - 存储型XSS（上传HTML/JS文件）。

柔狗安全内部培训资料

    - 服务器存储空间耗尽（DoS攻击）。
    - 敏感信息泄露（上传配置文件如`.env`）。
- **利用条件**：
    - 文件保存路径不可访问（如非Web目录）。
    - 服务器未解析上传的文件（如禁止执行脚本）。

任意文件上传GetShell（RCE via File Upload）

- **定义**：攻击者上传可执行文件（如PHP/JSP）并**直接触发远程代码执行（RCE）**。
- **CVE案例**：
    - **CVE-2023-23752**（Joomla!）：上传.php文件后可直接访问执行。
    - **CVE-2021-41773**（Apache HTTP Server）：结合路径遍历上传WebShell。
- **危害**：
    - 完全控制服务器（执行系统命令）。
    - 横向渗透内网。
    - 数据泄露或篡改。
- **利用条件**：
    - 文件被保存到Web可访问目录（如`/var/www/html/`）。
    - 服务器解析脚本（如Apache解析PHP）。

任意文件上传原理分析

我们发现不管是任意文件上传还是任意文件上传rce，关键点在于我们上传的后缀可控，因为我们上传什么格式后缀，就会以什么文件格式解析(有解析环境情况下)，所以我们审计的要点也是关注文件后缀的获取，只要后缀我们可控就是任意文件上传

JAVA中的任意文件上传利用

存在jsp解析环境

上传jsp🐎，getshell

如果能目录穿越的情况下可以覆盖关键文件

无jsp解析环境

上传html造成xss(水洞神器)

如果能目录穿越的情况下可以覆盖关键文件

关于spring boot中的jsp限制

在大多数项目都是基于SpringBoot架构进行的开发，官方不建议 SpringBoot 使用 JSP，并且做了一些限制。

具体官方文档：`https://docs.spring.io/spring-boot/docs/2.1.1.RELEASE/reference/htmlsingle/#boot-features-jsp-limitations`

柔狗安全内部培训资料

在spring boot中使用jsp的话需要引入相关依赖，自建WEB-INF,web.xml等操作，然后在pom.xml文件中导入解析依赖

```
<!--用于编译jsp-->
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
    <scope>provided</scope>
</dependency>
```

如果pom.xml中存在对应依赖，也就可能存在jsp解析，就可以尝试上🐂

文件检测

文件检测分为白名单和黑名单

如果是黑名单，列如限制了jsp等等

绕过方法：

1.使用其它可解析后缀

`.jsp/.jspa/.jspx/.jsw/.jsv/.jspf/.jtml/.jSp/.jSpx/.jSpa/.jSw/.jSv/.jSpf/.jHtml`

2.使用截断

`::$DATA/%00`

挖掘思路

白盒

文件上传关键字

```
uploda
//上传
new File
File
FileUpload
FileUploadBase
FileItemIteratorImpl
FileItemStreamImpl
FileUtils
UploadHandleServlet
FileLoadServlet
FileOutputStream
DiskFileItemFactory
MultipartRequestEntity
MultipartFile
com.oreilly.servlet.MultipartRequest
...
```

黑盒

上传常见功能点

柔狗安全内部培训资料

```
用户头像
订单信息图片
模板文件上传
文章演示图片
网站首页图片
后台文件操作
...
```

实战案例

jpress-v4.2.0

Tmall

Echo-v2.3

柔狗安全内部培训资料